

CSE 331: Computer Security

Fundamentals

Spring 2026

Race Conditions

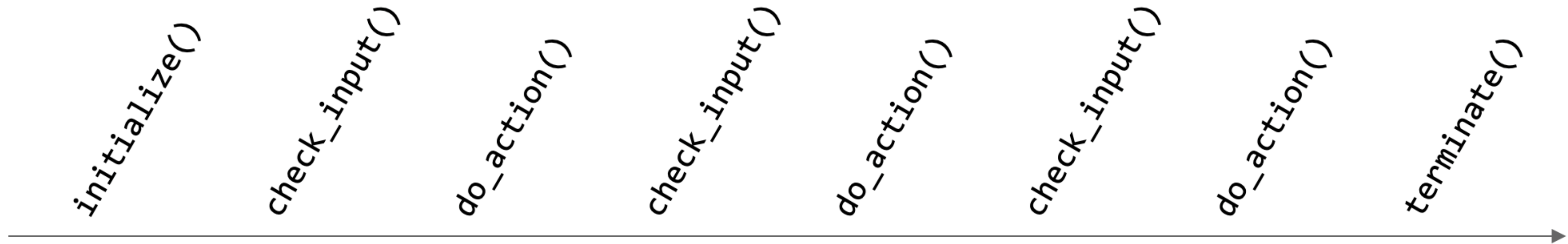
Amir Rahmati
Xinrui Yu



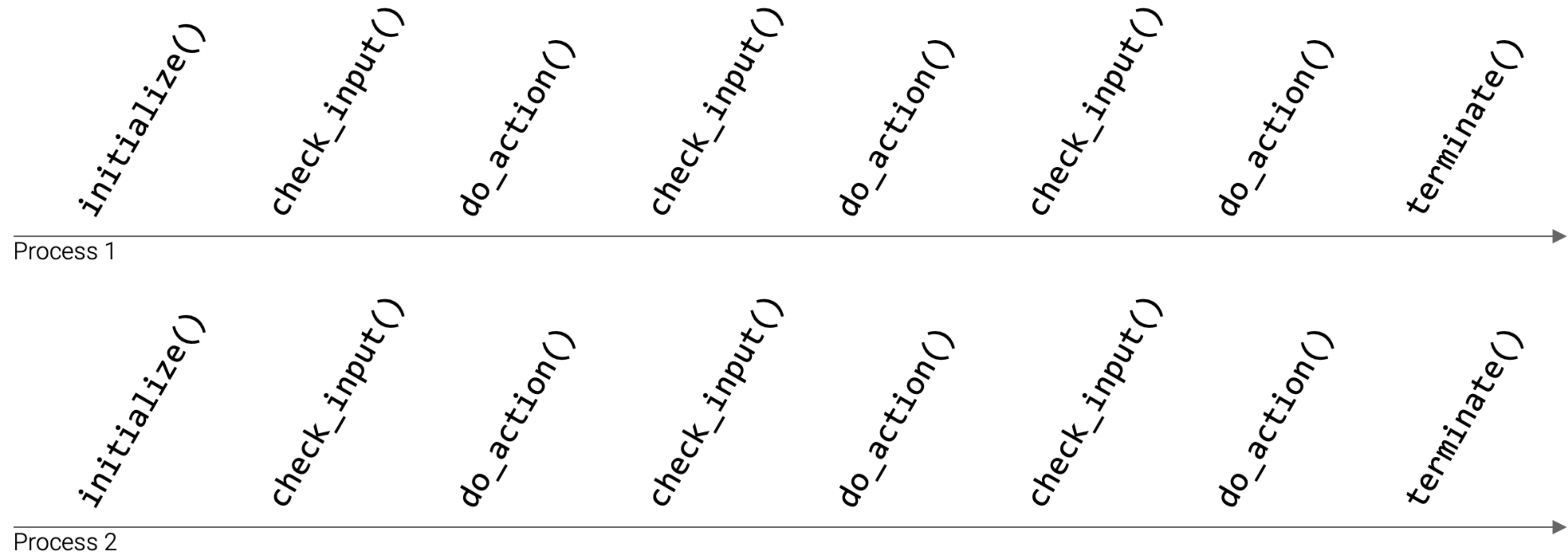
Stony Brook Institute
at Anhui University

安徽大學紐約石溪學院

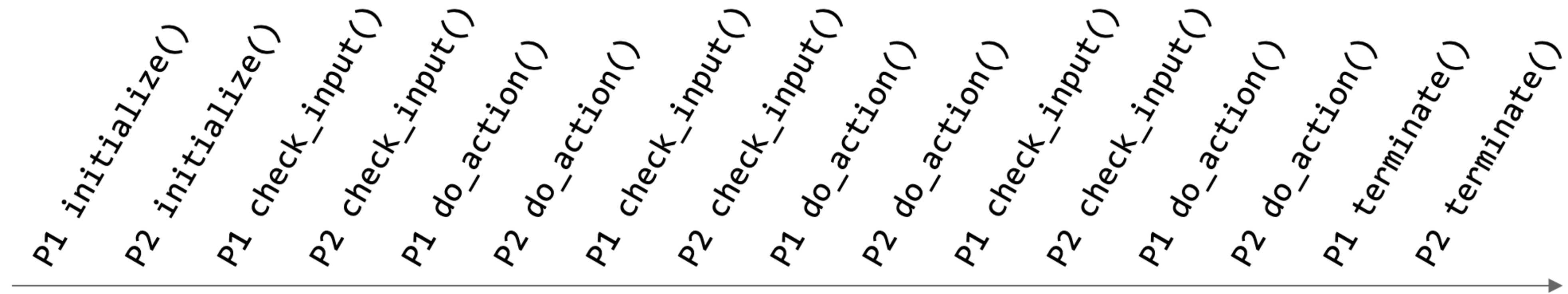
Process Execution



Process Execution



Process Execution



Concurrency

- **Old days:** single core CPUs!
- **Modern times:**
 - Multi-core, but still fewer cores than processes.
 - kernel decides what processes to "schedule" when
 - Limited-channel memory controllers (i.e., quad-channel memory)
 - Limited-channel storage media
 - Single-channel network communication
- **Bottom line:** Bottlenecks in computing architecture cause concurrent events to be at least partially serialized.

Order of Execution

- Without implicit dependencies or explicit effort by the program, execution order is only guaranteed within a process (really, within a thread).

P1 initialize()
P2 initialize()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 terminate()
P2 terminate()

P1 initialize()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()
P2 initialize()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 terminate()

P1 initialize()
P1 check_input()
P2 initialize()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()

P1 initialize()
P2 initialize()
P1 check_input()
P2 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 terminate()

P2 initialize()
P1 initialize()
P1 check_input()
P2 check_input()
P2 do_action()
P2 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 terminate()
P2 terminate()

TOCTOU: Time of Check / Time of Use

- Some execution orderings can be buggy!
- P1's do_action() might be taking actions in a changed world from the one examined by check_input()!

```
P1 initialize()
P2 initialize()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 terminate()
P2 terminate()
```

```
P1 initialize()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()

P2 initialize()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 terminate()
```

```
P1 initialize()
P1 check_input()
P2 initialize()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 terminate()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()
```

```
P1 initialize()
P2 initialize()
P1 check_input()
P2 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P1 terminate()
P2 do_action()
P2 check_input()
P2 do_action()
P2 check_input()
P2 do_action()
P2 terminate()
```

```
P2 initialize()
P1 initialize()
P1 check_input()
P2 check_input()
P2 do_action()
P2 check_input()
P1 do_action()
P1 check_input()
P1 do_action()
P2 do_action()
P1 check_input()
P2 check_input()
P1 do_action()
P2 do_action()
P1 terminate()
P2 terminate()
```

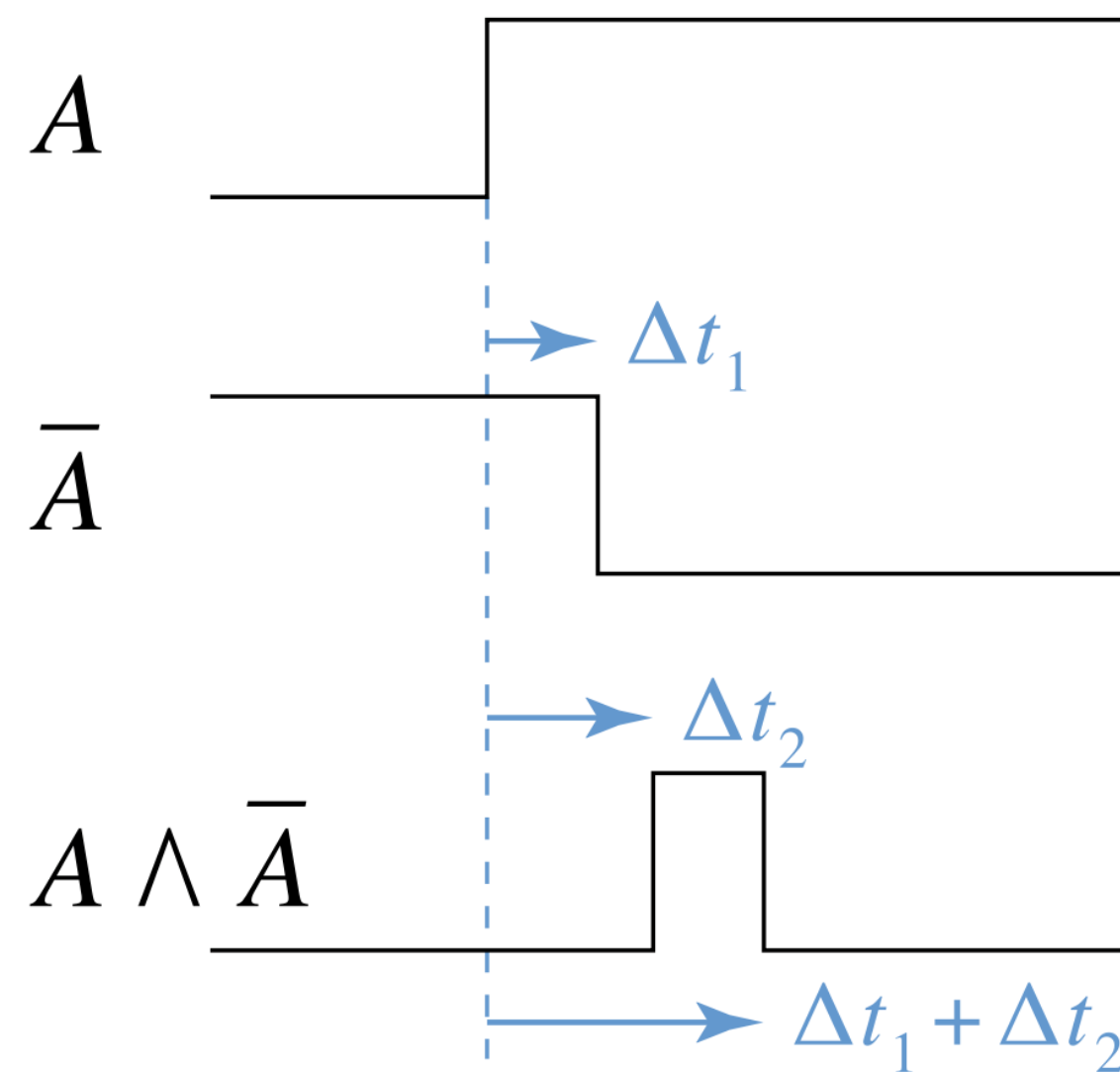
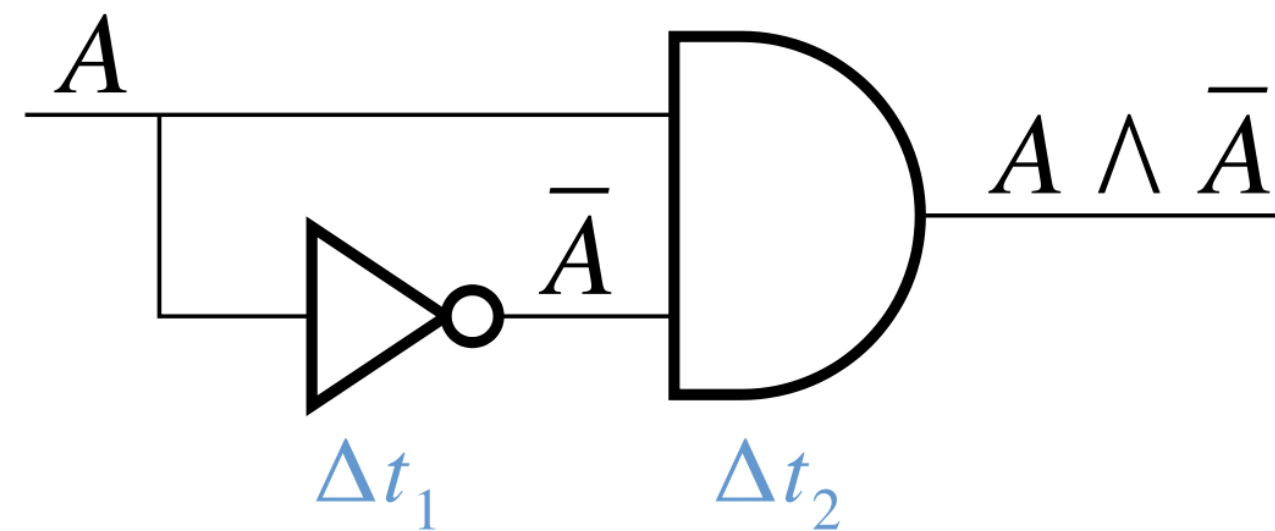
Race Condition

- How to abuse concurrency errors?
- Racing to carefully impact the state of an application during a weak point.
- Hence: Race Condition



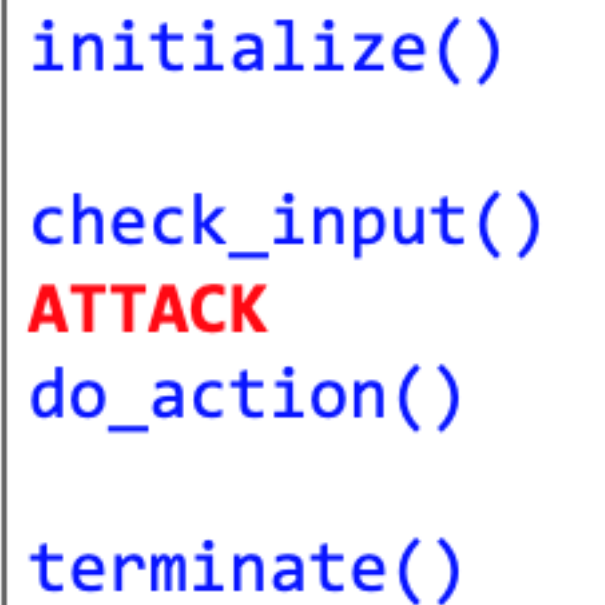
```
P1 initialize()  
  
P1 check_input()  
WEAK POINT  
P1 do_action()  
WEAK POINT  
P1 check_input()  
WEAK POINT  
P1 do_action()  
WEAK POINT  
P1 check_input()  
WEAK POINT  
P1 do_action()  
  
P1 terminate()
```


Race Condition



Exploiting Race Conditions

- Attackers exploit a race condition by **changing the state** that a program is running in while the program assumes that this state has not changed.
- To exploit a race condition, attackers must be able to impact said environment.
- One common case are **Races in the Filesystem**.



```
initialize()  
check_input()  
ATTACK  
do_action()  
terminate()
```

The Filesystem

- The filesystem is a part of the program state that the attacker can often influence.
- Consider:

```
01 int main(int argc, char **argv) {  
02     int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0755);  
03     write(fd, "#!/bin/sh\nnecho SAFE\n", 20);  
04     close(fd);  
05     execl("/bin/sh", "/bin/sh", argv[1], NULL);  
06 }
```

- Huge window of opportunity: any point between `open()` and full startup of `/bin/sh` is an attack window.
- How do we exploit it?
- Replacing the file between the time of creation and execution.
while /bin/true; do cp ATTACK inputfile; done

Upping the Challenge

- What if we make it more difficult?
- Consider:

```
01 int main(int argc, char **argv) {  
02     int echo_fd = open("/bin/echo", O_RDONLY);  
03     int fd = open(argv[1], O_WRONLY | O_CREAT, 0755);  
04     sendfile(fd, echo_fd, 0, 1024*1024);  
05     close(fd);  
06     execl(argv[1], argv[1], "SAFE", NULL);  
07 }
```

- Much smaller window of opportunity.
- How do we exploit it?
- General idea: slow down the victim as much as possible to make the vulnerable window open longer.

Being nice

- One method: nice
- The Linux kernel's scheduler supports different priorities for different programs. The nice command (and nice() system call) expose this functionality.
- Intensive commands might normally interfere with the rest of your system:

 \$ /usr/bin/intensive_command
- Using nice, they can be made to behave better:

 \$ nice /usr/bin/intensive_command
- nice is for CPU, while ionice is for I/O. Using them in combination can slow down programs enough to win races!

Path Complexity

- Filesystem races involve filesystem access, and not all filesystem access is equal... For example:

```
$ cat my_file
```

- Is much faster than:

```
$ cat a/b/c/d/e/f/g/h/i/j/k/l/m/n/o/p/q/r/s/t/u/v/w/x/y/z/my_file
```

- Why? The kernel takes time to go into all these directories!
- We can pass in SUPER long paths to slow the program down (Linux has a path size limit of 4096 bytes).

Filesystem Mazes

- Can we do better than 4096 bytes?

- YES, by using symbolic links!

- /my/maze/a/1/2/3/4/5/6/7/8/9/10/11/12/13/14/15/16/17/18/.../root
a_end
b/1/2/3/4/5/6/7/8/9/10/11/12/13/14/15/16/17/18/.../root
b_end
c/1/2/3/4/5/6/7/8/9/10/11/12/13/14/15/16/17/18/.../root
c_end
d/1/2/3/4/5/6/7/8/9/10/11/12/13/14/15/16/17/18/.../root
d_end
-

- Then we can use paths like /my/maze/a_end/root/b_end/root/c_end/root/d_end/root/some_file to reference /my/maze/some_file through a huge number of directory traversals.
- Note: Linux has a limit of 40 symbolic links per path resolution.

TOCTOU

- These types of race conditions tend to show up as Time of Check to Time of Use vulnerabilities (TOCTOU / TOCTTOU). Example: CVE-2019-7307

- The culprit: Ubuntu's crash reporting program (apport)

- The bug:

```
if not os.access(ifpath, os.R_OK) or os.path.getsize(ifpath) == 0:
    # create a document from scratch
    dom = xml.dom.getDOMImplementation().createDocument(None, 'apport', None)
else:
    try:
        dom = xml.dom.minidom.parse(ifpath)
    except ExpatError as e:
        raise ValueError('%s has invalid format: %s' % (_ignore_file, str(e)))
```

- The exploit:
 - Make sure ifpath is accessible to trigger the else condition.
 - Quickly symlink ifpath to something else.
- Can be used to read arbitrary files on the filesystem!

Mitigations

- Safer programming practices (`O_NOFOLLOW`, `mkstemp()`, etc).
- Symlink protections in `/tmp`
 - root cannot follow symlinks in `/tmp` that are owned by other users
 - specifically made to prevent these sorts of issues

Process —> Threads

- So far in this class, we have talked about processes.
- Processes have their own:
 - Virtual memory
 - stack
 - heap
 - ...
 - Registers
 - File descriptors
 - Process ID
 - Security properties
 - uid
 - gid
 - seccomp rules
- A process can have multiple threads.
- Threads share:
 - Virtual memory
 - File descriptors
- But have their own:
 - Registers
 - Stack
 - Thread ID
 - Security properties
 - uid
 - gid
 - seccomp rules
- Every process has at least its "main" thread.

Creating a Thread

- Threads can be created and managed using many different high-level libraries.
- Example: pthread

```
void *thread_main(int arg) {  
    printf("Thread %d, PID %d, TID %d, UID %d\n", arg, getpid(), gettid(), getuid());  
}
```

```
main() {  
    pthread_t thread1, thread2;  
    pthread_create(&thread1, NULL, thread_main, 1);  
    pthread_create(&thread2, NULL, thread_main, 2);  
    printf("Main thread: PID %d TID %d UID %d\n", getpid(), gettid(), getuid());  
    pthread_join(thread1, NULL);  
    pthread_join(thread2, NULL);  
}
```

- Execution order between threads is not deterministic!

Going Deeper

- At a low level, threads are created using the clone() system call.
- clone() is the successor of fork(), allowing for more control over what is shared between the parent and child.
- The pthread_create() library function uses clone() system call to create a child process that shares memory and other resources with the parent:

```
clone(  
child_stack = 0x7f81afdc7fb0,  
flags = CLONE_VM|           // parent and child will run in the same memory space  
      CLONE_FS|             // parent and child will share filesystem info (chroot(), chdir(), and similar effects will be shared between parent and child)  
      CLONE_FILES|          // parent and child will share file descriptors  
      CLONE_SIGHAND|        // parent and child will share signal handlers  
      CLONE_THREAD|         // signify that the child is a fellow thread of the parent  
      CLONE_SYSVSEM|        // parent and child will share semaphore information  
      CLONE_SETTLS|         // set a unique "Thread Local Storage" area for the new thread  
      CLONE_PARENT_SETTID|  // store the new thread ID at the memory location pointed to by the tid arg below  
      CLONE_CHILD_CLEARTID, // zero out the thread ID at the location pointed to by the tid arg below when the child exits  
parent_tid=[1926535],  
tls=0x7f81afdc8700,  
child_tidptr=0x7f81afdc89d0  
)
```

- clone() can do other things (i.e., starting containers with CLONE_NEWNS).

Discrepancies

- There are some discrepancies between libc (and libpthread) and the Linux system call interface. Some examples:
- **setuid:** The libc syscall wrapper for setuid() will set the UID of all the threads of the process, but the setuid() system call itself only sets the UID of the caller thread!
- **exit:** The libc syscall wrapper for exit() will actually call the exit_group() system call to terminate all the threads, but the exit() system call itself only terminates the caller thread!

Terminating Threads

- Threads are often long-running, and a common practice is to communicate using global variables.

```
int done = 0;
void *thread_main(int arg) {
    while (!done) {
        printf("Thread %d, PID %d, TID %d, UID %d\n", arg, getpid(), gettid(), getuid());
        sleep(1);
    }
}

main() {
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, thread_main, 1);
    pthread_create(&thread2, NULL, thread_main, 2);
    printf("Main thread: PID %d TID %d UID %d\n", getpid(), gettid(), getuid());
    getchar();
    done = 1;
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
}
```

- Might it be unsafe to access global memory from multiple threads?

Can there be races outside of the filesystem?

- The filesystem is a common race condition target because it is a shared resource between processes.
- Memory is a shared resource between threads.
- Race conditions in memory are rampant in multi-threaded programs

Motivating Example

memory-based TOCTOU data race

- Consider the following:

```
unsigned int size = 42;
void read_data() {
    char buffer[16];
    if (size < 16) {
        printf("Valid size! Enter payload up to %d bytes.\n", size);
        printf("Read %d bytes!\n", read(0, buffer, size));
    }
    else printf("Invalid size %d!\n", size);
}

void *thread_allocator(int arg) {
    while (1) read_data();
}

main() {
    pthread_t allocator;
    pthread_create(&allocator, NULL, thread_allocator, 0);
    while (size != 0) read(0, &size, 1);
    exit(0);
}
```

- This is a typical TOCTOU: the main thread can update size between the check and the read()!

Special case: Double Fetch

- Recall `copy_from_user()` in kernel space. Sometimes, kernel developers make mistakes:

```
int check_safety(char *user_buffer, int maximum_size) {  
    int size;  
    copy_from_user(&size, user_buffer, sizeof(size));  
    return size <= maximum_size;  
}
```

```
static long device_ioctl(struct file *file, unsigned int cmd, unsigned long user_buffer) {  
    int size;  
    char buffer[16];  
    if (!check_safety(user_buffer, 16)) return;  
    copy_from_user(&size, user_buffer, sizeof(size));  
    copy_from_user(buffer, user_buffer+sizeof(size), size);  
}
```

- This is a TOCTOU, with the race between the kernel and a sibling thread of the caller!

Other Data Races

- General data races can have weird effects:

```
unsigned int num = 0;
void *thread_main(int arg) {
    while (1) {
        num++;
        num--;
        if (num != 0) printf("NUM: %d\n", num);
    }
}
```

```
main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_main, 0);
    pthread_create(&t2, NULL, thread_main, 0);
    getchar();
    exit(0);
}
```

- No atomicity - Effects of code can get clobbered



```
num == 0
T1: mov rdi, [num] ; 0
T1: inc rdi ; 1
T1: mov [num], rdi ; 1
num == 1
T2: mov rdi, [num] ; 1
T2: inc rdi ; 2
T2: mov [num], rdi ; 2
num == 2
T1: mov rdi, [num] ; 2
T2: mov rdi, [num] ; 2
T1: dec rdi ; 1
T2: dec rdi ; 1
T1: mov [num], rdi ; 1
T2: mov [num], rdi ; 1
num == 1
```


Preventing Data Races

- You can prevent data races by carefully utilizing mutexes (inter-thread locks).

```
pthread_mutex_t lock;
unsigned int num = 0;

void *thread_main(int arg) {
    while (1) {
        pthread_mutex_lock(&lock);
        num++;
        num--;
        if (num != 0) printf("NUM: %d\n", num);
        pthread_mutex_unlock(&lock);
    }
}

main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread_main, 0);
    pthread_create(&t2, NULL, thread_main, 0);
    getchar();
    exit(0);
}
```

- A block of code protected by mutexes is called a critical section.

Detecting Data Races

- There is no magic bullet for detecting race conditions, but some tools exist!
- **valgrind** has two such tools: helgrind and drd, both of which detect data races if the relevant code is triggered by test cases.
- Academia is busy working on prototypes:
 - Dr Checker (<https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/machiry>) - Static Analysis
 - Razzer (<https://lifeasageek.github.io/papers/jeong-razzer.pdf>) - Fuzzer
 - KRACE (<https://www.cc.gatech.edu/~mxu80/pubs/xu:krace.pdf>) - Fuzzer
- In general, this problem remains unsolved. Another example: CVE-2020-12652, a double-fetch bug in an ioctl handler in the Linux kernel

Signals and Reentrancy

- Send a signal to a process:

```
int kill(pid_t pid, int sig)
```

SIGHUP	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	Term	Interrupt from keyboard
SIGQUIT	Core	Quit from keyboard
SIGILL	Core	Illegal Instruction
SIGABRT	Core	Abort signal from <code>abort(3)</code>
SIGFPE	Core	Floating-point exception
SIGKILL	Term	Kill signal
SIGSEGV	Core	Invalid memory reference
SIGPIPE	Term	Broken pipe: write to pipe with no readers; see <code>pipe(7)</code>
SIGALRM	Term	Timer signal from <code>alarm(2)</code>
SIGTERM	Term	Termination signal
SIGUSR1	Term	User-defined signal 1
SIGUSR2	Term	User-defined signal 2
SIGCHLD	Ign	Child stopped or terminated
SIGCONT	Cont	Continue if stopped
SIGSTOP	Stop	Stop process
SIGTSTP	Stop	Stop typed at terminal
SIGTTIN	Stop	Terminal input for background process
SIGTTOU	Stop	Terminal output for background process

Handling Signals

- Processes can register signal handlers to handle signals.

```
sighandler_t signal(int signum, sighandler_t handler)
```

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact)
```

- **Effect:** signals pause process execution and invoke the handler...
- **Access:** you can send ANY signal to ANY process that has the same rUID as you, even if their eUID is 0.
- **Capability:** this means that you can cause any program to suddenly and unexpectedly divert execution to the signal handler!
- This also works to yank programs out of a "critical section".

Offensive Signals

- Crazy issues that can arise when code runs in unexpected order!

```
int num = 0;
void signal_handler(int signum) {
    num = 0;
}

int main() {
    signal(SIGUSR1, signal_handler);
    while (1) {
        if (num == 0) num++;
        num--;
        if (num != 0) printf("NUM: %d\n", num);
    }
}
```



Reentrancy

- **Reentrant** functions are functions that would operate properly even when interrupted with an instance of themselves.

```
01 int tmp;  
02 void swap(int* x, int* y) {  
03     tmp = *x;  
04     *x = *y;  
05     *y = tmp;  
06 }  
07  
08 void call_swap() {  
09     int x = 1, y = 2;  
10     swap(&x, &y);  
11 }  
12  
13 int main() {  
14     signal(SIGUSR1, call_swap);  
15     call_swap();  
16 }
```

- What happens if this program receives SIGUSR1 before line 5?
- None of these functions are reentrant!

Reentrancy

- Do not call non-reentrant functions in your signal handlers:
 - Your handler might have interrupted those functions mid-execution.
 - Another signal might interrupt your signal handler's non-reentrant invocations mid-execution!
 - Depending on settings (SA_NODEFER flag to sigaction()), another iteration of the same signal might interrupt your signal!
- Note: malloc() and free() are not reentrant.
- man signal-safety for more info.